

```
SecFilter create[[:space:]]+table
SecFilter update.+set.+=
SecFilter union.+select
SecFilter or.+1[[:space:]]*= [[:space:]]1
SecFilter ',+..
SecFilter xp_enumsdn
SecFilter xp_cmdsshell
SecFilter xp_regread
SecFilter xp_regwrite
SecFilter xp_regdeletekey
```

The last five are particularly used for MS SQL server-specific injection attacks.

The only problem with *SecFilter* is that it scans the whole request instead of particular fields. Here, *SecFilterSelective* is useful; it allows you to choose exactly what to search. The syntax is:

```
SecFilterSelective LOCATION KEYWORD [ACTIONS]
```

Here, *LOCATION* decides which area of the request to be filtered. Hence, for SQL injection, you can also use:

```
SecFilterSelective SCRIPT_FILENAME "login.php" chain
SecFilterSelective ARG_user "!^[a-zA-Z0-9\.\@]{1,10}$"
```

The above code will validate the user parameter, and allow only the white-list of characters we have given. If for some reason you cannot take this approach, and must use a 'deny-what-is-bad' method, then at least remove single quotes ('), semicolons (;), dashes, hyphens (-), and parenthesis ("()").

XSS attacks

For XSS attacks, we can use the following directives:

```
SecFilter "<[.\n]+>"
SecFilter "<[[:space:]]*"script"
SecFilter "<script"
SecFilter "<.+>"
```

And also, some additional filters like:

```
SecFilterSelective THE_REQUEST "<[<meta">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<style">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<script">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<iframe">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<object">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<img">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<applet">?<[>]*">"
SecFilterSelective THE_REQUEST "<[<form">?<[>]*">"
```

Though these filters will detect a large number of XSS

attacks, they are not foolproof. Due to the multitude of different scripting languages, it is possible for an attacker to create many different methods for implementing an XSS attack that would bypass these filters. Hence, here it is advised that you also keep on adding your own filters.

To protect against an XSS attack done via PHP session cookies, you can use the following:

```
SecFilterSelective ARG_PHPSESSID "!^[0-9a-z]*$"
SecFilterSelective COOKIE_PHPSESSID "!^[0-9a-z]*$"
```

Command execution attacks

For command execution attacks, you can use the following directives:

```
SecFilter /etc/passwd
SecFilter /bin/lis
```

Here, the attacker may try to use a string like */bin/.sh* to bypass the filter—but *mod_security* automatically reduces */./* to */and* *//* to */*, and also decodes URL-encoded characters. You can also use the white-list approach:

```
SecFilterSelective SCRIPT_FILENAME "directory.php" chain
SecFilterSelective ARG_dir "!^[a-zA-Z/_\.\@-0-9]*$"
```

This chained rule-set will only allow letters, numbers, underscore, dash, forward slash, and period in the *dir* parameter. Filtering out command directory names is also a good option, and can be done as follows:

```
SecFilterSelective THE_REQUEST "/^(etc|bin|sbin|tmp|var|opt|dev|kernel)$/"
SecFilterSelective ARGS "bin/"
```

Session fixation

During session fixation, in one of its phases, the attacker needs to somehow inject the desired session ID into the victim's browser. We can mitigate these issues by implementing the following:

```
# Weaker XSS protection, but allows common HTML tags
SecFilter "<[[:space:]]*"script"
# Prevent XSS attacks (HTML/Javascript injection)
SecFilter "<.+>"
# Block passing Cookie/Session IDs in the URL
SecFilterSelective THE_REQUEST "(document\.cookie|Set-Cookie|SessionId=)"
```

Directory traversal attacks

For path/directory traversal attacks, the following directives are mostly used: